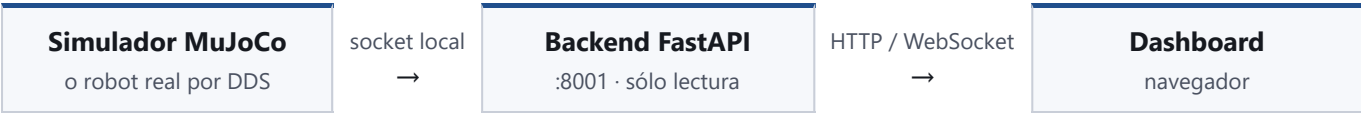


1. ARQUITECTURA

El sistema es cliente-servidor con una única frontera: la API HTTP/WebSocket del backend de la cátedra, que es de sólo lectura y no se modificó.



Endpoints. GET /info en el arranque, para conocer la forma del robot: cantidad de motores, nombres y apoyos. GET /telemetria en polling, o WS /ws en push, para los datos. Los endpoints parciales (/motores, /imu, /bms, /fuerzas) no se usan: /telemetria ya trae los cuatro subsistemas en un frame coherente, y pedirlos por separado sería una foto de cada panel en un instante distinto.

Estructura del frontend. Cuatro capas, sin framework. js/main.js es el único que conoce todas las piezas a la vez: ningún panel sabe de dónde vienen los datos, y ningún transporte sabe qué se dibuja.

CAPA	RESPONSABILIDAD
Configuración	direcciones, umbrales, ventanas
Transporte	hablar con el backend y emitir eventos
Estado	historia circular y muestras capturadas
Presentación	un archivo por panel, más los gráficos

Patrones aplicados

- **Observer.** Los transportes emiten telemetry y status; los paneles se suscriben. Es lo que permite cambiar de polling a WebSocket en caliente sin tocar un solo panel.
- **Strategy.** PollingTransport y SocketTransport implementan la misma interfaz (start/stop más los dos eventos) y son intercambiables.
- **Adapter.** El backend normaliza Go2 (12 motores) y G1 (29) al mismo JSON; el frontend agrega su propia adaptación por forma de robot, armando la tabla y los apoyos desde /info y no de constantes.
- **Facade.** /telemetria como interfaz única de los cuatro subsistemas.

2. DECISIONES DE DISEÑO

Sin framework y sin build. El TP no lo exige y sumarlo complica la entrega: el dashboard se abre con doble clic. Se verificó que el backend responde Access-Control-Allow-Origin: * incluso ante

Origin: null, que es lo que manda un archivo abierto desde el disco. Por eso los módulos son scripts clásicos con un namespace y no módulos ES, que por file:// estarían bloqueados.

Las filas de la tabla se crean una sola vez. Después sólo cambia el texto y el color de cada celda. Reconstruir 29 filas × 6 celdas diez veces por segundo es trabajo de DOM tirado a la basura, hace imposible seleccionar el texto y pierde la posición del scroll en cada frame.

El gráfico nunca se recrea. El Chart se instancia una vez; cada frame agrega un punto por serie, descarta el más viejo al pasar de 300 muestras (30 s a 10 Hz) y redibuja sin animación. Los datasets se rebobinan sólo cuando cambia la *selección* de motores, y la serie nueva se siembra con la historia que ya tiene el store, así no arranca desde una línea vacía. Con 29 curvas superpuestas el gráfico no comunica nada, de modo que se grafican hasta seis series elegidas con checkboxes.

Umbrales editables. Los valores por defecto son los del enunciado (40 y 60 °C), pero son configurables porque la temperatura simulada tiene un techo de 55 °C y la banda roja no se puede ejercitar de otra forma.

Estado del enlace como panel propio. Dos señales: la que reporta el transporte, y un *watchdog* que marca el enlace caído si el último frame tiene más de 2 s, aunque el socket siga abierto. Se conservan los últimos valores en pantalla, pero señalizados, nunca en silencio: telemetría que puede estar vieja sin decirlo es peor que telemetría ausente.

CSV armado en el navegador. Las columnas son la *unión* de las claves de todas las filas capturadas, no las de la primera: una sesión puede tener filas de formas distintas —otro robot, otra cantidad de celdas— y quedarse con la primera descartaría columnas sin avisar. Se exporta con BOM UTF-8 y CRLF para que Excel abra bien los encabezados con acentos.

Robustez. Si Chart.js no carga desde el CDN, los paneles numéricos siguen funcionando y se avisa; si /info falla, el dashboard lo dice y reintenta en lugar

de asumir que hay un Go2 del otro lado; si llega un frame de otro robot, se descarta; y un frame al que le falta un subsistema muestra rayas en los valores ausentes, nunca ceros.

3. POLLING VS. WEBSOCKET PARA ESTE CASO

Medición propia · 15 s por modo, mismo backend en --demo, localhost

	REST POLLING 500 MS	WEBSOCKET
Frames recibidos	30 (2,0 Hz)	148 (9,9 Hz)
Latencia	RTT media 5,4 ms · p95 8,7 ms · máx 20,3 ms	intervalo entre frames: media 101,9 ms · p95 103,1 ms
Tráfico	2,8 kB/s	13,8 kB/s
Payload por frame	1 400 B (Go2) · 3 140 B (G1)	igual

Latencia. El RTT del polling es bajísimo en localhost, pero no es la métrica que importa: lo que importa es la *antigüedad* del dato en pantalla. Con un intervalo de 500 ms, un valor puede tener hasta 500 ms más el RTT, y en promedio 250 ms. Con el WebSocket la antigüedad media es de unos 50 ms, la mitad del período de 100 ms con que publica el backend. Es un factor 5 y no depende de la red: es el intervalo fijo.

Muestras perdidas. El backend publica a 10 Hz. Con polling cada 500 ms se lee 1 de cada 5 frames: el 80 % de los datos se descarta antes de llegar al gráfico. Para una tabla de temperaturas da igual —la temperatura se mueve lento— pero para el torque y para el estado de apoyos, que cambian con la fase de la marcha, el polling no ve transiciones: el apoyo de una pata puede cambiar y volver a cambiar entre dos requests. Los mismos 300 puntos del búfer cubren 30 s por WebSocket y 150 s por polling, así que además la ventana deja de ser la de 30 s que pide el enunciado.

Carga. El WebSocket mueve 4,9 veces más bytes, pero es una sola conexión y un solo handshake: el polling paga cabeceras HTTP completas dos veces por

segundo, y en el servidor cada request es una entrada de log, una ejecución de handler y un ciclo de conexión. A 10 Hz por polling —lo que haría falta para igualar al socket— serían 10 requests/s por cliente, y el backend está compartiendo la máquina con MuJoCo.

Complejidad. El polling son cinco líneas y no puede fallar de formas raras: si un request se cae, el siguiente lo arregla. El WebSocket necesita reconexión con backoff, detección de socket abierto pero mudo y —según se verificó— una dependencia extra en el servidor (websockets). Acá son unas 40 líneas más el watchdog.

Conclusión. Para telemetría de robot el WebSocket es la opción correcta: los datos son un flujo continuo a frecuencia conocida, la fuente decide cuándo hay algo nuevo y el consumidor quiere todo. El polling es preferible cuando el dato cambia poco, cuando el consumidor quiere controlar la frecuencia, o cuando hay proxies e infraestructura que no toleran conexiones largas. El dashboard implementa los dos y arranca en polling, que es el modo que funciona con la instalación mínima que indica el TP.

Las diferencias verificadas entre el enunciado y el entorno de la cátedra, con su evidencia, están en el anexo docs/hallazgos-del-entorno.md.